

Del diagrama al código

Los dos criterios con que se corrige un modelo — cohesión y acoplamiento— y la traducción de cada caja del diagrama a una clase de Python.

Rubén Schnettler • Docente

INACAP Valparaíso • Analista Programador

BLOQUE 4 DE 8

RECUPERACIÓN

Dónde quedamos



PROGRAMAR CON OBJETOS

Clase, atributos, métodos, constructor y herencia, escritos y ejecutados.



LA NOTACIÓN UML

Notación UML, las cuatro relaciones y la multiplicidad, en diagrams.net.



EL CASO DE LA EVALUACIÓN

EcoTech Solutions y cómo se consulta a la IA sin entregar lo que devuelve.



EL ORDEN DE ESTE BLOQUE

Primero los **dos criterios** con que se juzga un modelo: cohesión y acoplamiento. Después, la traducción de cada caja del diagrama a **código Python**.

Un diagrama que no se puede escribir en código es un diagrama que aún no está terminado.

PRIMERA PARTE

CÓMO SE JUZGA UN MODELO

Cohesión y acoplamiento

Los criterios técnicos con que se corrige un modelo

CRITERIO TÉCNICO 1

Cohesión: una clase, un tema



COHESIÓN

Qué tan **relacionado entre sí** está todo lo que hay dentro de una clase. Alta cohesión significa que todo lo que contiene sirve al mismo propósito.



COHESIÓN BAJA

Empleado: registrarHoras(),
enviarCorreo(), respaIdarBaseDatos(),
generarInformePDF()

Cuatro temas distintos en una sola clase.



COHESIÓN ALTA

Empleado: registrarHoras(),
obtenerHorasDelMes(),
actualizarSalario()

Todo trata del empleado y nada más.

CRITERIO TÉCNICO 2

Acoplamiento: cuánto depende de otra



ACOPLAMIENTO

Cuánto necesita una clase **saber de las otras** para funcionar. Conviene que sea bajo: así un cambio no obliga a modificar media aplicación.



ACOPLAMIENTO ALTO

Si Departamento conoce la estructura interna de RegistroTiempo, cualquier cambio en el registro rompe el departamento.



ACOPLAMIENTO BAJO

Departamento le pregunta a Empleado por sus horas, y no le importa cómo las guarda por dentro. **Eso lo permite el encapsulamiento.**

Regla de la industria: **alta cohesión, bajo acoplamiento.**

LO QUE EXIGE EL CRITERIO 1.1.4

Tres principios de diseño, en la mayoría de las clases

Responsabilidad única

Cada clase tiene **una sola razón para cambiar**. Si se modifica por dos motivos distintos, conviene separarla.

Encapsulamiento

Atributos privados y acceso por métodos. En el diagrama se ve en los signos - y +.

Cohesión

Todo lo que hay en la clase trata del mismo asunto. Se comprueba leyendo la lista de métodos.



PARA LLEGAR A EXPERTO

No basta aplicarlos: hay que **dejar registro documental** de dónde se aplicó cada uno. Una tabla de tres filas en el informe resuelve el indicador 1.1.4.I.17.

EJEMPLO DE AJUSTE

La clase que hacía de todo



LO QUE PROPUSO LA IA

SistemaGestion

```
- empleados: list
- proyectos: list
- departamentos: list

+ agregarEmpleado()
+ asignarProyecto()
+ registrarHoras()
+ generarInforme()
+ autenticar()
+ cifrarDatos()
```



DESPUÉS DEL AJUSTE

Empleado
registrarHoras()

Departamento
asignarEmpleado()

Proyecto
asignarEquipo()

Informe
generar()

Usuario
autenticar()

La justificación que va al informe: «se eliminó SistemaGestion porque concentraba seis responsabilidades distintas; sus métodos se reasignaron a la clase dueña de cada dato». Esa frase vale más que el dibujo.

EJERCICIO 1 • EN PAREJA

8 minutos

¿Alta o baja cohesión?

Tres clases propuestas por una herramienta de IA. Para cada una, decida si la cohesión es **alta o baja** y, si es baja, **qué método se va y a qué clase**.

Ticket

- + crear()
- + cerrar()
- + enviarCorreoAlCliente()
- + respaldarBaseDatos()

Sensor

- + leerTemperatura()
- + calibrar()
- + ultimaLectura()

Usuario

- + autenticar()
- + cambiarClave()
- + generarInformeVentas()



LA PREGUNTA QUE DECIDE

¿Todos los métodos **tratan del mismo tema**? Si uno habla de otra cosa, pertenece a otra clase.

RESPUESTA • EJERCICIO 1 Revisión en plenario

Qué se queda y qué se va

CLASE	COHESIÓN	QUÉ SE HACE
Ticket	BAJA	enviarCorreoAlCliente() se va a Notificador y respaldarBaseDatos() a RespaldoBD: ninguno de los dos habla del ticket
Sensor	ALTA	Los tres métodos tratan de medir. No se toca
Usuario	BAJA	generarInformeVentas() se va a Informe: la identidad de una persona no tiene nada que ver con las ventas



LA FRASE QUE VA AL INFORME

«Se retiró generarInformeVentas() de Usuario porque introducía una responsabilidad ajena a la clase; se reasignó a Informe, dueña de ese dato.» **Esa justificación es la que se evalúa**, no el dibujo.

SEGUNDA PARTE

DEL DIAGRAMA AL CÓDIGO

De UML a Python

El diagrama se convierte en código. Cada caja del diagrama pasa a ser una clase escrita.

ANTES DE EMPEZAR

Por qué se escribe código en esta parte



LO QUE DICE LA RÚBRICA

El último indicador pide **defender la viabilidad técnica del diseño definitivo en Python**. No se puede defender lo que nunca se intentó escribir.



LO QUE NO SE EVALÚA ESTA UNIDAD

Que el sistema funcione completo, ni la base de datos, ni la interfaz. Eso viene más adelante.



LO QUE SÍ CONVIENE TENER

Las clases del diagrama escritas y ejecutando, para comprobar que el modelo **es realizable**.

Escribir el código revela errores del diagrama que dibujando no se ven. Es la mejor revisión posible del modelo.

ORGANIZACIÓN DEL PROYECTO

La estructura de carpetas del proyecto

```
ecotech/  
  .venv/  
  src/  
    empleado.py  
    departamento.py  
    proyecto.py  
    registro_tiempo.py  
  main.py  
  diagramas/  
    ecotech-uml-v1.png  
    ecotech-uml-v2.png
```



UNA CLASE POR ARCHIVO

Es la convención de Python: nombre del archivo en minúscula con guion bajo, nombre de la clase en mayúscula inicial.



ESTA ESTRUCTURA SE REUTILIZA

Es la misma carpeta con el entorno virtual que se creó al comenzar la unidad. Más adelante se convierte en un repositorio de GitHub.

TERCERA PARTE

EL CÓDIGO, LÍNEA POR LÍNEA

El código, línea por línea

De la caja UML a la clase Python

LA TRADUCCIÓN

Cada parte del diagrama tiene su equivalente

EN EL DIAGRAMA UML	EN PYTHON
Nombre de la clase	<code>class Empleado:</code>
Atributo con - (privado)	<code>self.__salario</code> · dos guiones bajos delante
Atributo con + (público)	<code>self.nombre</code> · sin guiones
Método con +	<code>def registrar_horas(self, ...):</code>
Agregación o asociación	Una lista dentro de la clase: <code>self.empleados = []</code>
Generalización (herencia)	<code>class Gerente(Empleado):</code>

Los nombres de método cambian de estilo: en UML se escribe registrarHoras; en Python la convención es registrar_horas.

EJERCICIO 2 · INDIVIDUAL

12 minutos

Traducir una caja a Python

Proyecto

```
- codigo: str
- nombre: str
- horasPresupuestadas: float
- horasUsadas: float

+ registrarAvance(horas: float): float
+ estaSobrePlazo(): bool
```



Escriba la clase completa

- El constructor, con los atributos privados
- Los dos métodos, con su return
- Un objeto creado y una consulta impresa



DOS AVISOS

horasUsadas parte en cero: **no va entre los parámetros.**
Y los nombres pasan a snake_case.

RESPUESTA · EJERCICIO 2

Revisión en plenario

La caja Proyecto, escrita en Python

```
class Proyecto:
    def __init__(self, codigo, nombre, horas_presupuestadas):
        self.__codigo = codigo
        self.__nombre = nombre
        self.__horas_presupuestadas = horas_presupuestadas
        self.__horas_usadas = 0

    def registrar_avance(self, horas):
        self.__horas_usadas += horas
        return self.__horas_usadas

    def esta_sobre_plazo(self):
        return self.__horas_usadas > self.__horas_presupuestadas
```

- 1 Los cuatro atributos del piso del medio pasan a `self.__`, en el mismo orden.
- 2 `horas_usadas` **no está en los paréntesis**: parte en cero y crece con el uso.
- 3 `registrar_avance` devuelve `float`: el total acumulado. Por eso lleva `return`.
- 4 `esta_sobre_plazo` devuelve `bool` porque la comparación `>` ya vale verdadero o falso: no hace falta ningún `if`.

ESCRIBA ESTO

Empleado • el constructor

src/empleado.py

```
class Empleado:
    """Un trabajador de EcoTech."""

    contador = 0

    def __init__(self, nombre, correo, salario):
        Empleado.contador += 1
        self.id = Empleado.contador
        self.nombre = nombre
        self.correo = correo
        self.__salario = salario
        self.registros = []
```

- 1 **contador** es de la clase, no del objeto: sirve para **asignar el identificador automáticamente**, como pide el caso
- 2 **__init__** es el constructor: se ejecuta al crear cada objeto
- 3 **self** es el objeto que se está creando. Va siempre como primer parámetro
- 4 Los dos guiones bajos de **__salario** lo hacen **privado**: es el - del diagrama

ESCRIBA ESTO

Empleado • métodos y encapsulamiento

src/empleado.py • continuación

```
def obtener_salario(self):  
    return self.__salario  
  
def actualizar_salario(self, nuevo):  
    if nuevo ≤ 0:  
        raise ValueError("Salario inválido")  
    self.__salario = nuevo  
  
def registrar_horas(self, registro):  
    self.registros.append(registro)  
    return True
```

- 5 El salario **no se lee directamente**: se pide por un método. Así se controla quién accede
- 6 Y tampoco se escribe directamente: al pasar por un método **se puede validar**. Aquí está el requisito de validación de entradas del caso
- 7 Esto es **encapsulamiento** en la práctica: el dato protegido y el acceso controlado

ESCRIBA ESTO

RegistroTiempo y la composición

src/registro_tiempo.py

```
class RegistroTiempo:
    """Horas trabajadas en un proyecto."""

    def __init__(self, fecha, horas, descripcion):
        if horas <= 0 or horas > 24:
            raise ValueError("Horas fuera de rango")
        self.fecha = fecha
        self.horas = horas
        self.descripcion = descripcion
```

- 8 La validación va **en el constructor**: así es imposible crear un registro imposible
- 9 Nótese que **RegistroTiempo no guarda a qué empleado pertenece**: es el empleado quien tiene la lista
- 10 Eso es la **composición** del diagrama: el registro vive dentro del empleado y no tiene existencia propia

ESCRIBA ESTO

Gerente y la herencia

src/gerente.py

```
from empleado import Empleado

class Gerente(Empleado):
    """Empleado que dirige un departamento."""

    def __init__(self, nombre, correo, salario, area):
        super().__init__(nombre, correo, salario)
        self.area = area

    def aprobar_horas(self, registro):
        return registro.horas ≤ 12
```

- 11 El paréntesis en `Gerente(Empleado)` es el triángulo del diagrama
- 12 `super().__init__()` llama al constructor del padre: nombre, correo y salario **no se repiten**
- 13 Solo se agrega lo propio del gerente. Esto es evitar duplicación de código, como pide el caso

ESCRIBA ESTO

El programa principal

main.py

```
from src.empleado import Empleado
from src.registro_tiempo import RegistroTiempo

carla = Empleado("Carla Rojas", "crojas@ecotech.cl", 1450000)
r = RegistroTiempo("2026-08-26", 4, "Diseño UML")
carla.registrar_horas(r)

print(carla.id, carla.nombre)
print("Horas registradas:", len(carla.registros))
```

Al ejecutar con el botón de reproducción del editor:

```
$ python main.py
1 Carla Rojas
Horas registradas: 1
```



QUÉ DEMUESTRA ESTA SALIDA

Que el identificador se asignó solo y que **el modelo es realizable**. Eso es lo que hay que afirmar en la defensa.

COMPROBACIÓN

Demostrar que el encapsulamiento funciona

En el intérprete

```
>>> carla.__salario
AttributeError: 'Empleado' object has
no attribute '__salario'

>>> carla.obtener_salario()
1450000

>>> carla.actualizar_salario(-100)
ValueError: Salario inválido
```



ESTA CAPTURA SIRVE PARA EL INFORME

Es la **evidencia** de que el encapsulamiento no es una declaración: está implementado y se comprueba.



PARA LA DEFENSA ORAL

Una pregunta probable: «¿cómo protege los datos personales?». Mostrar esta salida responde mejor que cualquier explicación.

ACTIVIDAD INDIVIDUAL 60 minutos

Escribir las clases del diagrama propio

1. Crear la carpeta y el entorno virtual
2. Escribir **al menos tres clases** del diagrama ajustado
3. Dejar los atributos sensibles privados, con su método de acceso
4. Incluir **una validación** en algún constructor
5. Escribir `main.py` creando dos objetos
6. Ejecutar y capturar la salida de la terminal



SI EL DIAGRAMA Y EL CÓDIGO NO COINCIDEN

Corregir **el diagrama**. Si algo no se puede escribir, el modelo estaba mal, y ese hallazgo va al informe como mejora fundamentada.



SIN BASE DE DATOS NI INTERFAZ

Eso viene después. Aquí basta con que las clases existan y se puedan instanciar.

CUARTA PARTE

PREPARACIÓN DE LA DEFENSA

Explicar el modelo en voz alta

Un tercio de la rúbrica se evalúa en defensa oral.
Lo que no se puede explicar en voz alta casi
nunca está bien entendido.

PREPARACIÓN

Preguntas que conviene tener respondidas

¿Por qué RegistroTiempo es composición y no agregación?

¿De dónde sacó la multiplicidad de esa relación?

¿Cómo protege el sistema los datos personales de los empleados?

¿Qué le sugirió la IA que usted decidió no hacer, y por qué?

¿Qué pasaría con su modelo si un empleado pudiera estar en dos departamentos?

¿Este diseño es realizable en Python? Muéstrelo.

Cómo se responde bien: citando el enunciado del caso y no la intuición. «Porque el requisito dice que un empleado pertenece a un solo departamento a la vez» es completa; «porque me pareció más lógico» no lo es.

SIMULACIÓN EN PAREJAS

40 minutos

Simular la defensa

En parejas, veinte minutos por persona. Uno presenta con su diagrama a la vista; el otro escucha siguiendo la rúbrica.

1. Presentar los cuatro bloques, cronometrando
2. El compañero anota **cada afirmación sin fundamento**
3. El compañero hace **tres preguntas** de la lista anterior
4. Intercambiar roles
5. Corregir en el informe lo que no se supo explicar



POR QUÉ EN VOZ ALTA

Lo que no se puede explicar en voz alta **casi nunca está bien entendido**. La simulación detecta esos puntos a tiempo.



DATO PRÁCTICO

Los tropiezos de la simulación suelen coincidir con los párrafos más débiles del informe. Conviene revisar justamente esos.

RESPUESTA · SIMULACIÓN DE LA DEFENSA

Referencia

Tres respuestas modelo

¿Por qué RegistroTiempo es composición y no agregación?

«Porque un registro de tiempo no tiene existencia propia: identifica una jornada de un empleado determinado. Si ese empleado sale del sistema, sus registros dejan de significar algo. El enunciado lo dice cuando establece que cada registro pertenece a un empleado.»

¿Cómo protege el sistema los datos personales?

«El salario y el correo son atributos privados: - en el diagrama y dos guiones bajos en Python. Ninguna otra clase los toca de forma directa; el acceso pasa por métodos públicos que validan quién pregunta.»

¿Qué le sugirió la IA que usted decidió no hacer?

«Propuso una clase SistemaGestion con seis responsabilidades. Se rechazó por cohesión baja: cada método pertenecía a la clase dueña de su dato. Se redistribuyeron en cinco clases y el acoplamiento bajó.»

Lo que hunde una respuesta: «me pareció más lógico» · «así lo hace todo el mundo» · «lo generó la herramienta y quedó bien».

TALLER • REVISIÓN CON EL DOCENTE

45 minutos

Última revisión antes de entregar

Trabajo individual en el computador. El docente revisa por puesto y responde consultas puntuales.



PRIORIDAD DE LA REVISIÓN

Primero los **números** de la lista de verificación. Después la redacción. Un informe bien escrito al que le falta un requerimiento pierde más que uno correcto con prosa mejorable.

Errores que se ven todos los semestres

- El diagrama exportado ilegible
- Falta la versión inicial del modelo
- Los prompts resumidos en vez de textuales
- Análisis de la IA sin decir qué se decidió
- Sin matriz de trazabilidad
- Sin conclusiones

CIERRE DEL BLOQUE

Con qué se cierra la unidad

Lo que queda cubierto

- Cohesión y acoplamiento como criterios de revisión
- Los tres principios que exige el criterio 1.1.4
- La traducción de cada caja del diagrama a una clase

Lo que corresponde hacer

- Cerrar el informe con la matriz de trazabilidad
- Revisar la lista de verificación antes de subirlo
- Simular la defensa con el diagrama a la vista



EL CRITERIO QUE ORDENA TODO LO ANTERIOR

Un diagrama que no se puede escribir en código está incompleto, y un modelo que no se puede explicar en voz alta no se distingue de uno copiado.

FIN DEL BLOQUE

Del diagrama al código, y del código a la explicación

El modelo, el código y el argumento son tres formas de la misma decisión de diseño.

TI3V21 · Programación Orientada a Objeto Seguro · INACAP Valparaíso